NAME – Aatir Saad

REG NO – 2411021061653

Problem Name - Maximum Perimeter Triangle

Problem Statement - Given an array of stick lengths, use of them to construct a non-degenerate triangle with the maximum possible perimeter. Return an array of the lengths of its sides as integers in non-decreasing order.

If there are several valid triangles having the maximum perimeter:

1. Choose the one with the *longest maximum side*.
2. If more than one has that maximum, choose from them the one with the *longest minimum side*.
3. If more than one has that maximum as well, print any one them.

Problem HackerRank Link - https://www.hackerrank.com/challenges/maximum-perimeter-triangle/problem

Github Submission Link – https://github.com/aatir-saad/DAA-hackerrank/tree/main

Solution Steps -

Step 1: Sort the Numbers

Arrange all elements in ascending order.

Reason:

- Makes it easier to check the triangle condition
- Reduces 3 conditions to just 1

Step 2: Check Consecutive Triplets

Consider every group of three consecutive elements:

(a[i], a[i+1], a[i+2])

Step 3: Start from the Largest Values

Traverse from the end of the sorted array toward the beginning.

Reason:

- Larger numbers are more likely to satisfy the triangle condition
- Ensures we find a valid triangle quickly

Step 4: Apply Triangle Condition

For each triplet:

a[i] + a[i+1] > a[i+2]

- If true → triangle is possible
- If false → move to the next triplet

Step 5: Output Result

- If any valid triplet is found → print those three values
- If no triplet satisfies the condition → output -1

Complete JAVA Code –

```
import java.io.ByteArrayInputStream;

import java.io.IOException;

import java.io.InputStream;

import java.io.PrintWriter;

import java.util.Arrays;

import java.util.InputMismatchException;


public class B {

  InputStream is;

  PrintWriter out;

  String INPUT = "";


  void solve()

  {

    int n = ni();
```

```
    int[] a = na(n);
    Arrays.sort(a);
    for(int i = n-3;i >= 0;i--){
        if(a[i]+a[i+1] > a[i+2]){
            out.println(a[i] + " " + a[i+1] + " " + a[i+2]);
            return;
        }
    }
    out.println(-1);
}

void run() throws Exception
{
    is = INPUT.isEmpty() ? System.in : new ByteArrayInputStream(INPUT.getBytes());
    out = new PrintWriter(System.out);

    long s = System.currentTimeMillis();
    solve();
    out.flush();
    if(!INPUT.isEmpty())tr(System.currentTimeMillis()-s+"ms");
}

public static void main(String[] args) throws Exception { new B().run(); }

private byte[] inbuf = new byte[1024];
private int lenbuf = 0, ptrbuf = 0;

private int readByte()
{
    if(lenbuf == -1)throw new InputMismatchException();
```

```
        if(ptrbuf >= lenbuf){
            ptrbuf = 0;
            try { lenbuf = is.read(inbuf); } catch (IOException e) { throw new InputMismatchException();
}
            if(lenbuf <= 0)return -1;
        }
        return inbuf[ptrbuf++];
    }

    private boolean isSpaceChar(int c) { return !(c >= 33 && c <= 126); }
    private int skip() { int b; while((b = readByte()) != -1 && isSpaceChar(b)); return b; }

    private double nd() { return Double.parseDouble(ns()); }
    private char nc() { return (char)skip(); }

    private String ns()
    {
        int b = skip();
        StringBuilder sb = new StringBuilder();
        while(!(isSpaceChar(b))){ // when nextLine, (isSpaceChar(b) && b != ' ')
            sb.appendCodePoint(b);
            b = readByte();
        }
        return sb.toString();
    }

    private char[] ns(int n)
    {
        char[] buf = new char[n];
        int b = skip(), p = 0;
```

```
    while(p < n && !(isSpaceChar(b))){
        buf[p++] = (char)b;
        b = readByte();
    }
    return n == p ? buf : Arrays.copyOf(buf, p);
}

private char[][] nm(int n, int m)
{
    char[][] map = new char[n][];
    for(int i = 0;i < n;i++)map[i] = ns(m);
    return map;
}

private int[] na(int n)
{
    int[] a = new int[n];
    for(int i = 0;i < n;i++)a[i] = ni();
    return a;
}

private int ni()
{
    int num = 0, b;
    boolean minus = false;
    while((b = readByte()) != -1 && !((b >= '0' && b <= '9') || b == '-'));
    if(b == '-'){
        minus = true;
        b = readByte();
    }
```

```
    while(true){
        if(b >= '0' && b <= '9'){
            num = num * 10 + (b - '0');
        }else{
            return minus ? -num : num;
        }
        b = readByte();
    }
}

private long nl()
{
    long num = 0;
    int b;
    boolean minus = false;
    while((b = readByte()) != -1 && !((b >= '0' && b <= '9') || b == '-'));
    if(b == '-'){
        minus = true;
        b = readByte();
    }

    while(true){
        if(b >= '0' && b <= '9'){
            num = num * 10 + (b - '0');
        }else{
            return minus ? -num : num;
        }
        b = readByte();
    }
```

```
    }

    private static void tr(Object... o) { System.out.println(Arrays.deepToString(o)); }
}
```

Observation/Solution -

The key observation in this problem is that the triangle condition (sum of any two sides greater than the third) can be simplified by first sorting the array in ascending order. Once sorted, for any three elements $a \leq b \leq c$, it is sufficient to check only whether $a + b > c$, since the other two conditions will automatically hold. Another important insight is that if a valid triangle exists, it will most likely be found among consecutive elements in the sorted array, so there is no need to check all possible triplets. By starting from the largest elements and moving backward, we increase the chances of quickly finding a valid triangle. If a triplet fails the condition, then any smaller elements will also fail, making further checks unnecessary for that position. This observation reduces the complexity from a brute-force approach to an efficient solution using sorting and a single traversal.